

1

If $P_A = P_G$ we can just model this as $P_A = P_G = P/2$

Parallel time is $T_P \frac{P_A}{p/2} + P_G \rightarrow \frac{P/2}{p/2} + P/2 = P/p + P/2 = P(1/2 + 1/p)$

By Gustafson's law, since P_G scales with number of processors then serial time must also scale the computations of P_G

P_A on the other hand is constant amount of work so remains as P_A

So serial time is $T_S = P_A + (p/2)P_G \rightarrow P/2 + pP/4 = P(1/2 + p/4)$

$$\text{Speedup} = \frac{T_S}{T_P} = \frac{P(1/2 + p/4)}{P(1/2 + 1/p)} = \frac{1/2 + p/4}{1/2 + 1/p} = \frac{1 + p/2}{1 + 2/p} = \frac{p + p^2/2}{p + 2} = \frac{2p + p^2}{2p + 4}$$

$$\text{Speedup} = \frac{2p + p^2}{2p + 4} = O(p) \text{ as } p \text{ approaches } \infty$$

2

a

4Ghz with 1FLOP per cycle means one process would have a performance of 4GFLOP. So 4GLFOP per processor, and total performance is 640GFLOP so you would need $640/4 = 160$ processors to get that performance

b

Using roofline model, first machine balance is $640GFLOP/64Gbps = 10$

Thus since $8 < 10$ the algorithm would be memory bound thus peak performance would be $8 * 64GLFOP = 512GLFOP$

Then minimum performance is if the algorithm only uses one processor, from part a that means then min performance is 4GLFOP

Thus: $4GLFOP \leq \text{Performance} \leq 512GLFOP$

c

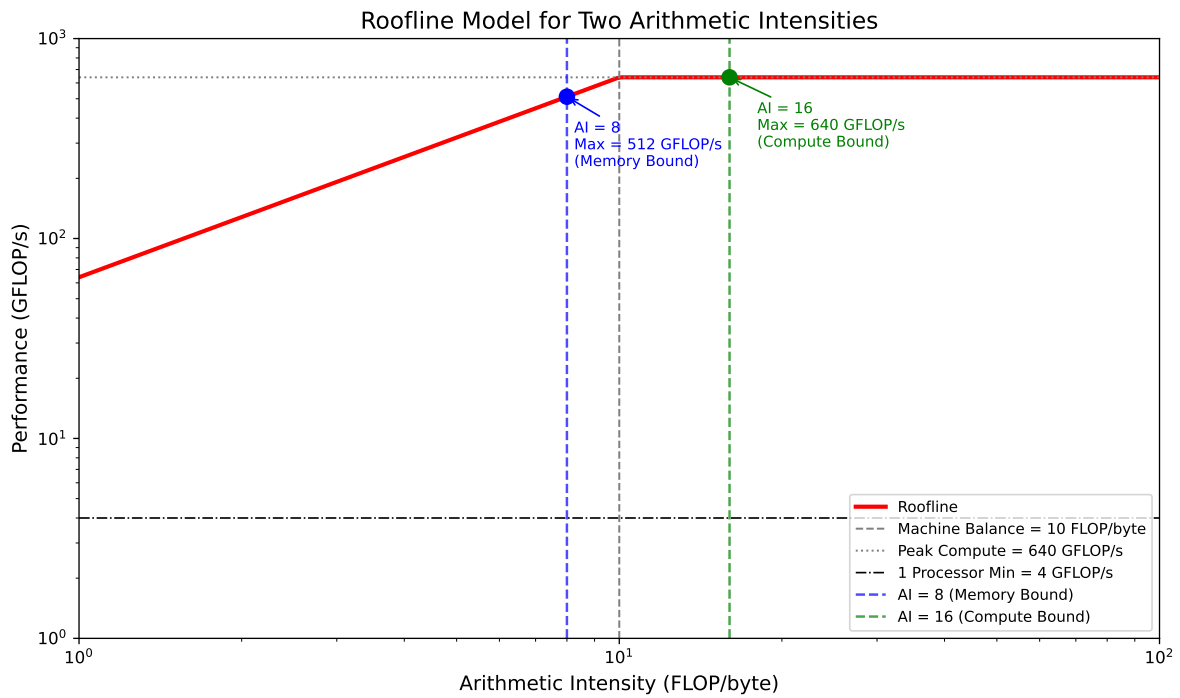
Using roofline model, first machine balance is $640GFLOP/64Gbps = 10$

Thus since $16 > 10$ the algorithm would be compute bound thus peak performance would be $640GFLOP$

Then minimum performance is if the algorithm only uses one processor, from part *a* that means then min performance is $4GFLOP$

Thus: $4GFLOP \leq Performance \leq 640GFLOP$

d



3

3.1

a

```
// In thread bid, tid

// Copy this block's partition into shared memory (2 elements per thread)
Fetch A[2*(bid*p + tid)]      -> Sh[2*tid]
Fetch A[2*(bid*p + tid) + 1] -> Sh[2*tid + 1]
Syncthreads()

for (i = 0 to 2p-1) {
    if (i % 2 == 0) {
        l: Read Sh[2*tid]
        r: Read Sh[2*tid+1]

        if (l > r) {
            temp = l
            Sh[2*tid] = r
            Sh[2*tid+1] = temp
        }
    }
    else {
        if (tid != 0) {
            l: Read Sh[2*tid-1]
            r: Read Sh[2*tid]

            if (l > r) {
                temp = l
                Sh[2*tid-1] = r
                Sh[2*tid] = temp
            }
        }
    }
    Syncthreads()
}

// Store the sorted partition back to DRAM
```

```
Store Sh[2*tid]    -> A[2*(bid*p + tid)]
Store Sh[2*tid+1] -> A[2*(bid*p + tid) + 1]
```

This results in S sorted partitions of A , so the entire array isn't sorted but we have sorted sub arrays.

b

$T_P = O(1)$ fetch + $O(2p)$ parallelized comparisons + $O(1)$ store so $O(p)$

The parallel time is $O(p)$ but serialized time would be $O(p^2)$ ($2p$ loop $\times p$ comparisons per loop) so this is a scalable solution because $\text{Speedup} = \frac{p^2}{p} = O(p)$ so scalable.

c

Cost $C = T_p \times p = O(p) \times p = O(p^2)$

Since $C = O(p^2) = T_S$ the problem is cost optimal

Note that best serial time is $O(p \log p)$ so this is not cost optimal against the BEST serial sort but cost optimal when compared to its serialized version

3.2

a

		A							
		0	1	2	3	4	5	6	7
		5	7	8	13	16	18	21	25
B	0 9	1	1	1	0	0	0	0	0
	1 11	1	1	1	0	0	0	0	0
	2 12	1	1	1	0	0	0	0	0
	3 17	1	1	1	1	1	0	0	0
	4 20	1	1	1	1	1	1	0	0
	5 22	1	1	1	1	1	1	1	0
	6 23	1	1	1	1	1	1	1	0
	7 27	1	1	1	1	1	1	1	1

Processor id	Diagonals / Ouptut Indices	Element Values / Output Values
0	0,1	5,7
1	2,3	8,9
2	4,5	11,12

Processor id	Diagonals / Ouput Indices	Element Values / Output Values
3	6,7	13,16
4	8,9	17,18
5	10,11	20,21
6	12,13	22,23
7	14,15	25,27

Final merge is $C = [5, 7, 8, 9, 11, 12, 13, 16, 17, 18, 20, 21, 22, 23, 25, 27]$

b

Diagonal indices are highlighted above. This diagonal is the one corresponding to the 7th index in C so it belongs to processor 3.

Let matrix M represent the 0,1 comparison matrix for the arrays A, B . Then we get the following elements / comparisons:

Indices (i, j)	Comparson $A[i]$ vs $B[j]$	M Value
0,7	25 vs 9	1
1,6	21 vs 11	1
2,5	18 vs 12	1
3,4	16 vs 17	0
4,3	13 vs 20	0
5,2	8 vs 22	0
6,1	7 vs 23	0
7,0	5 vs 27	0

c

Suppose the $1 - 0$ transition on a diagonal happens between $M[i][j]$ and $M[i - 1][j + 1]$. We distinguish the two scenarios by looking at the value of $M[i - 1][j]$ or the block that shares the corner with the transition blocks.

1. If $M[i][j - 1] = 1$ then we enter the intersection from the top

Thus we have the output $C[i + j] = B[i - 1]$

2. If $M[i][j - 1] = 0$ then we enter the intersection from the left

Thus we have the output $C[i + j] = A[j]$

In class we also reiterated explicitly the fact that because we only go right and down in our path (never back tracking), the above two statements will hold true.

Thus for diagonal 7, we find the intersection point lies between $M[3][4]$ and $M[2][5]$, this is because $B[3] = 17 > A[4] = 16$ but $B[2] = 12 \leq A[5] = 18$

Then we need to check $M[2][4]$ and since $B[2] = 12 \leq A[4] = 16$ then $M[2][4] = 0$ so we come from the left meaning we should push $A[4]$ to $C[3 + 4]$

Therefore we would store $A[4] = 16$ into $C[7]$ from the above steps and comparisons.

d

```
// In thread bid, tid    (one SMP, p threads, merging sorted A[0:p-1] and B[0:p-1] into C[0:2p-1])

// Copy both arrays into shared memory
Fetch A[tid] -> ShA[tid]
Fetch B[tid] -> ShB[tid]
Syncthreads()

for (j = 0 to 1) {
    k = 2*tid + j

    // Clip the diagonal to the p x p matrix (handles different diagonal lengths)
    lo = max(0, k+1-p)
    hi = min(k+1, p)

    // Binary search for the first 0 entry M[k-a][a] on diagonal k
    for (s = 0 to log_2(p)) {          // fixed # of steps, enough for the longest diagonal
        if (lo < hi) {
            mid = floor((lo + hi) / 2)
            if (ShA[mid] < ShB[k-mid]) {    // M[k-mid][mid]=1 -> transition is right
                lo = mid + 1
            }
            else {                          // M[k-mid][mid]=0 -> transition here or left
                hi = mid
            }
        }
    }
}

a = lo
b = k + 1 - a
```

```

// Which direction did the path come from (part c)
if (a == 0) {
    out = ShB[b-1]
}
else if (b == 0) {
    out = ShA[a-1]
}
else if (ShA[a-1] < ShB[b-1]) {    // M[b-1][a-1]=1: came from top
    out = ShB[b-1]
}
else {                            // M[b-1][a-1]=0: came from left
    out = ShA[a-1]
}

Store out -> C[2*tid + j]
}

```

On the topic of different length diagonals, just after the start of the first for loop before the binary search, we clip the *lo* and *hi* indices to ensure that they are in the correct spot. This ensures we never go out of bounds on either *A* or *B* at the start, and the different lengths of diagonals just change the number of steps we go through but our nested for loop ensures we reach the number of steps needed for the longest diagonal.

As for all 0 or all 1 diagonals, essentially the binary search part would end with *lo*=*hi* at either the lowest index or highest index, depending on if the algorithm is all 1 or all 0. In the case of all 0

e

Diagonal $k = 7$, $p = 8$, so $lo = 0, hi = 8$

Step	<i>lo</i>	<i>hi</i>	<i>mid</i>	Entry $M[7 - mid][mid]$	Compare $A[mid], B[7 - mid]$	Value	Update Values
1	0	8	4	$M[3][4]$	$A[4] = 16 < 17 = B[3]$	1	$lo = 5$
2	5	8	6	$M[1][6]$	$A[6] = 21 < 11 = B[1]$	0	$hi = 6$
3	5	6	5	$M[2][5]$	$A[5] = 18 < 12 = B[2]$	0	$hi = 5$

Then step 4 in the loop is skipped because now $lo \geq hi$ so intersection is found in 3 steps and is between $M[3][4]$ and $M[2][5]$

To get the subsequent direction you would need one more step to calculate $M[2][4]$ but this part only asks for finding the intersection which takes 3 steps

f

$T_P = O(1) \text{ fetch} + 2 \times O(\text{binary_search}) + O(1) \text{ store}$ so $O(\text{binary_search})$

Since binary search is $O(\log n)$ and our largest search is $n = 2p$ then $T_P = O(\log p)$

Serial time would be $O(p)$ since you can just do the merge part of merge sort basically to find that part's time complexity of $T_S = O(p)$

Thus Speedup $= \frac{T_S}{T_P} = \frac{O(p)}{O(\log p)}$ which comes out to $O(p)$ thus this is a scalable solution

g

Cost $C = T_p \times p = O(p \log p)$ so not cost optimal because setial merge is $O(p) < O(p \log p)$

h

Like we discussed in class, a processor whose condition doesn't hold is masked rather than running different code. All processors in the SMP step through the same instructions (Single Program), but when they reach a data dependent branch like `if (ShA[mid] < ShB[k-mid])` or the direction check, each processor evaluates the condition on its own data (Multiple Data). The `if` side then runs with only the processors where it was true active, and the others are masked off and do nothing. The `else` side then runs with the opposite set masked. This gets us our SPMD.

The same thing happens in the binary search loop. Every processor runs the loop the same fixed number of times, but once a processor has found its 1-0 transition (`lo == hi`) it is masked inside the `if (lo < hi)` for the rest of the loop, so finishing early causes no problems.

This works without extra synchronization because, after the one `Syncthreads()` that makes sure the shared memory copy is done, each processor only reads `ShA/ShB` and writes to its own 2 independent locations `C[2*tid]` and `C[2*tid+1]`. No processor has to wait on another's result. The cost of masking is that the branches are serialized, so the SMP takes as long as its slowest processor.

i

For each diagonal k , all p threads work on it together. The diagonal (including the imaginary $1 / 0$ ends) has at most $p + 1$ possible intersection points, so each thread checks the candidate points $lo + tid$ and $lo + tid + p$ (the second one only exists for the longest diagonal). A thread checks whether the cell before its point is a 1 and the cell after is a 0. Exactly one thread finds the 1-0 transition, and that thread checks the direction and writes $C[k]$.

```
// In thread bid, tid    (one SMP, p threads, merging sorted A[0:p-1] and B[0:p-1] into C[0:2p-1])

// Copy both arrays into shared memory
Fetch A[tid] -> ShA[tid]
Fetch B[tid] -> ShB[tid]
Syncthreads()

for (k = 0 to 2p-1) {                                // ALL threads work on diagonal k together
    lo = max(0, k+1-p)                                // possible intersection points a = lo .. hi
    hi = min(k+1, p)

    for (j = 0 to 1) {                                // thread tid checks points lo + tid and lo + tid + p
        a = lo + tid + j*p
        if (a <= hi) {
            b = k + 1 - a

            // Note M[b][a-1]=1, and M[b-1][a]=0

            left_is_1  = (a == lo) or (ShA[a-1] < ShB[b])
            right_is_0 = (a == hi) or (ShA[a] >= ShB[b-1])

            if (left_is_1 and right_is_0) { // only one thread finds the 1-0 transition
                if (a == 0) {                // came from top
                    out = ShB[b-1]
                }
                else if (b == 0) {            // came from left
                    out = ShA[a-1]
                }
                else if (ShA[a-1] < ShB[b-1]) { // M[b-1][a-1] = 1: came from top
                    out = ShB[b-1]
                }
                else {                        // M[b-1][a-1] = 0: came from left
                    out = ShA[a-1]
                }
            }
        }
    }
}
```

```

        }
        Store out -> C[k]
    }
}
}
Syncthreads()                // finish diagonal k before moving to k+1
}

```

j

$T_P = O(1)$ fetch + $2p \times O(1)$ comparisons + $O(1)$ store so $T_P = O(p)$

This means worse than the algorithm in part **d** because $O(p) > O(\log p)$. This makes intuitive sense because now we are not using the fact that we don't need a lot of the comparisons along the diagonal because we know the diagonal is sorted!

3.3

a

MERGE takes as inputs X, Y and diagonal indices $start, end$ indicating which diagonals this SMP is responsible for in the merge and outputs into a temp array. Note that $r - l$ should be the output length of T and in this problem $r - l = \text{len}(X) + \text{len}(Y)$

```

// In thread bid, tid    (S blocks of p threads, A = S sorted runs of 2p elements)

for (k = 1 to log_2(S)) {
    len = 2p * 2^(k-1)                // length of each run being merged

    if (bid % 2^k == 0) {              // S / 2^k active SMPs
        X = A[2*bid*p : 2*bid*p + len - 1]    // left run  (made by block bid)
        Y = A[2*bid*p + len : 2*bid*p + 2*len - 1]    // right run (made by block bid + 2^(k-1))
        T[2*bid*p : 2*bid*p + 2*len - 1] = MERGE(X, Y, 0, 2*len - 1)    // all 2*len outputs
    }

    GridSync()                        // barrier across ALL SMPs (in practice: one kernel launch)
    swap(A, T)                        // this iteration's output is next iteration's input
}
// A[0 : N-1] is now sorted

```

b

At iteration k , we know each processor does 2^{k+1} diagonals each of length $p \times 2^k$. Thus each iterations of $O(2^{k+1} \log(p \times 2^k))$

So we sum up the cost of each iteration, since there are $\log_2 S$ iterations: $T_P = \sum_{k=1}^{\log S} 2^{k+1} \log(p \times 2^k) = O(S \log(pS))$

c

Essentially the idea is very similar to 3.3a but instead of leaving each merge of X, Y to only a single SMP, we can do an output partition (because outputs are independent) but still have the length of the diagonals be growing. So we get the following [same MERGE inputs and outputs as 3.3a because I edited 3.3a to work better for this problem :)]

```
// In thread bid, tid    (S blocks of p threads, ALL active every iteration)

for (k = 1 to log_2(S)) {
    len = 2p * 2^(k-1)           // length of each run being merged
    base = (bid / 2^k) * 2*len    // start of the pair of runs this SMP helps merge
    o = (bid % 2^k) * 2p         // this SMP makes outputs o .. o+2p-
    1 of that merge

    X = A[base : base + len - 1] // left run
    Y = A[base + len : base + 2*len - 1] // right run
    T[base + o : base + o + 2p - 1] = MERGE(X, Y, o, o + 2p - 1) // only this SMP's 2p outputs

    GridSync()                  // barrier across all SMPs (one kernel launch per k)
    swap(A, T)
}
// A[0 : N-1] is now sorted
```

At iteration k , we know each processor does 2 diagonals each of length $p \times 2^k$. Thus each iterations of $O(2^{k+1} \log(p \times 2^k))$

So we sum up the cost of each iteration, since there are $\log_2 S$ iterations: $T_P = \sum_{k=1}^{\log S} \log(p \times 2^k) = O(\log S \log(pS))$, here we actually did the sum over everything bc the last term did not dominate this time.

Thus this is more efficient than the algorithm in 3.3a